

REGPOL CALLAO

Documentación Técnica del Sistema

Región Policial del Callao — UNITIC

Versión 1.0 — 23 de junio de 2026

1. Resumen del proyecto

REGPOL Callao es el portal web y sistema de gestión institucional de la Región Policial del Callao (UNITIC).

Incluye: portal público informativo, evaluaciones psicológicas MMPI-2, gestión de convenios y cursos, sorteos, CMS del portal y paneles administrativos por rol.

Repositorio: https://github.com/ANTHONY109823/REGPOL_CALLAO

Autor técnico: Ing. Anthony Ccayo — UNITIC — 2026

2. Arquitectura

Modelo híbrido desacoplado:

- Frontend estático: carpeta public/ desplegada en GitHub Pages.
- Backend API: Node.js + Express en Railway.
- Base de datos: PostgreSQL gestionada en Railway (variable DATABASE_URL).

URLs de producción:

- Portal web: GitHub Pages del repositorio (rama main).
- API backend: <https://regpolcallao-production.up.railway.app>

El archivo public/api-config.js detecta si el sitio corre en localhost o producción y apunta al backend correspondiente.

Health check del servidor: GET /health (usado por Railway).

3. Stack tecnológico

Backend: Node.js >= 18, Express 4.x, pg (PostgreSQL), cors, pdfkit.

Frontend: HTML5, CSS3, JavaScript vanilla (sin framework SPA).

Iconos: Font Awesome 6 (CDN).

PDF evaluaciones: pdf_gen.js (PDFKit) + mmpi2_score.py (cálculo MMPI-2 opcional).

CI/CD: GitHub Actions (.github/workflows/deploy-pages.yml) publica public/ en Pages al hacer push a main.

Backend deploy: Railway (railway.json, comando node server.js).

4. Base de datos — PostgreSQL

Motor: PostgreSQL. Conexión vía DATABASE_URL con SSL en producción.

Inicialización automática en initDB() al arrancar server.js (CREATE TABLE IF NOT EXISTS + ALTER COLUMN IF NOT EXISTS).

No hay archivos de migración separados; el esquema evoluciona inline en server.js.

Tablas principales:

- admins — Usuarios del panel (usuario, passhash SHA-256, rol, nombre, unidad, permisos JSONB).
- preguntas — 566 ítems del cuestionario MMPI-2 (numero, texto, activa, orden).
- evaluaciones — Evaluaciones enviadas o parciales (CIP, datos personales, respuestas JSONB, completada).

- progresos — Borradores guardados sin enviar (clave=CIP, respuestas JSONB, bloque_max, total_resp).
- divisiones — DIVOPUS 1, DIVOPUS 2, DIVOPUS 3, DIVUES.
- unidades_pol — Comisarías y unidades vinculadas a divisiones.
- configuracion — Clave-valor (p. ej. unidades_activas para habilitar evaluaciones).
- items_portal — Convocatorias de convenios y cursos.
- inscripciones — Inscripciones de personal a convocatorias.
- sorteos_portal / resultados_sorteo — Sorteos y ganadores publicados.
- portal_configuracion — CMS del portal (data_json con novedades, carrusel, reseña, etc.).

Índices: idx_eval_comisaria, idx_eval_unidad en evaluaciones.

Relaciones: inscripciones != items_portal; unidades_pol != divisiones; resultados_sorteo != sorteos_portal.

5. Roles y autenticación

Login: POST /admin/login != token SHA-256 almacenado en localStorage (regpol_session).

Cabecera de API autenticada: x-admin-token.

Cache de sesión en servidor: 5 minutos (authCache).

Roles:

- unitic — Super Admin: acceso total, dashboard general del sistema, usuarios, divisiones.
- bienestar — Acceso amplio a evaluaciones (redirige a panel-admin.html).
- usuario — Admin de área según permisos JSONB.

Permisos granulares: evaluaciones, descargas, cms_cursos, cms_convenios, cms_inicio, cms_resena, cms_labor, cms_novedades.

Rutas de panel: panel-admin.html (gestión) o panel-usuario.html (solo lectura/CMS limitado según permisos).

6. Módulos funcionales

Portal público (index.html, portal.js): inicio, novedades, reseña histórica, nuestra labor, unidades, convenios, cursos, carrusel de encabezado.

Evaluación MMPI-2 (evaluacion.html, evaluacion.js): registro por CIP, guardado de progreso, envío final a evaluaciones.

Panel admin (panel-admin.html): dashboard por rol, evaluaciones, preguntas, descargas PDF/CSV, CMS, convocatorias, inscripciones, sorteos, usuarios.

Panel usuario (panel-usuario.html): vista modular según permisos del operador.

Sorteo en vivo (sorteo-live.html): visualización pública de resultados.

7. API REST — Endpoints principales

Públicos (sin token):

GET /health, /config, /preguntas, /progreso, /portal/configuracion, /portal/items, /portal/sorteos, /unidades-publico

POST /guardar, /progreso, /portal/items/:id/inscribir, /admin/login

Autenticados (x-admin-token):

Evaluaciones: GET /evaluaciones, /stats, /listar, /descargar, /admin/avances, /admin/registro-cip

DELETE /admin/evaluaciones/:id, /admin/progresos?cip=, /admin/evaluaciones-lote

PDF: GET /pdf/efectivo, /pdf/grupo, /admin/preview-resultado, /admin/preview-avance

Admin sistema: GET /admin/stats-sistema (solo unitic), /admin/stats-gestion

Preguntas: CRUD /admin/preguntas

Usuarios/estructura: CRUD /admin/usuarios, /admin/divisiones, /admin/unidades

Portal gestión: /admin/configuracion, /admin/items, /admin/inscripciones, /admin/sorteos

PUT /config — activar dependencias para evaluaciones

8. Flujo de evaluaciones MMPI-2

1. El efectivo ingresa CIP en evaluacion.html; el sistema valida dependencia activa (config unidades_activas).
2. Durante el test, POST /progreso guarda borrador en tabla progresos.
3. Al finalizar, POST /guardar inserta/actualiza evaluaciones y elimina el progreso si completada=true.
4. Un CIP tiene un único registro en evaluaciones (upsert por CIP).
5. Administradores listan, filtran por división/unidad, ven resultados MMPI-2 y generan PDF.
6. Eliminación: individual (por fila), por unidad (lote) o limpieza total (solo Super Admin, confirmación ELIMINAR).

9. Estructura de archivos

server.js — Servidor Express, initDB, todos los endpoints.

pdf_gen.js — Generación de PDF individual y por grupo.

preguntas_data.json — Seed de 566 preguntas MMPI-2.

mmpi2_score.py — Script Python para puntuación MMPI-2.

public/ — Frontend estático completo.

public/api-config.js — URL del backend.

public/panel-admin.html — Panel principal de administración.

public/panel-cms.js — Lógica CMS compartida.

public/portal.js — Portal público y carrusel.

scripts/ — Utilidades de mantenimiento y generación.

railway.json — Configuración de despliegue Railway.

.github/workflows/deploy-pages.yml — CI para GitHub Pages.

10. Variables de entorno

PORT — Puerto del servidor (default 3000).

DATABASE_URL — Cadena de conexión PostgreSQL (obligatoria en producción).

Desarrollo local: npm start o INICIAR_SERVIDOR.bat; frontend en GitHub Pages local o servidor estático; api-config apunta a localhost:3000.

11. Optimizaciones recientes

Dashboard Super Admin: resumen general (/admin/stats-sistema) en lugar de solo evaluaciones.

Cache de estadísticas (/stats) 45 s; cache de configuración 120 s.

Eliminación de código legacy: panel-admin.js, panel-bienestar.html, respuestas_webapp.gs.

Endpoints DELETE para limpieza de evaluaciones y avances en base de datos.

Migraciones inline de columnas faltantes en progresos (unidad, cargo, sexo, armamento, foto, grado).

Timeouts y botón Reintentar en dashboards para reducir carga de CPU en navegador.

12. Seguridad y consideraciones

Contraseñas de admin almacenadas como SHA-256 (no bcrypt).

Tokens de sesión generados con crypto; validación por cabecera x-admin-token.

Filtro por unidad asignada para operadores sin permiso global de evaluaciones.

CORS habilitado para consumo desde GitHub Pages.

Datos sensibles (fotos, respuestas) en JSONB/TEXT en PostgreSQL; no exponer DATABASE_URL.

Recomendación: rotar contraseñas seed por defecto en producción.